

UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

Week 01 – Computing with real numbers

- 1 Computers
 - Binary computer
 - Arithmetic operations
 - IEEE extensions – `inf`, `NaN`
- 2 Errors
 - Absolute and relative errors, and number of significant figures
- Rounding error
- Catastrophic cancellation
- 3 Efficiency - Time
 - Floating-point operations
 - Estimating computation time
- 4 Risks
- 5 References and links

Binary computer – units

- Computers are **binary**: they store and operate on bits.
 - A **bit** is the smallest unit of digital information, represented by a single binary digit: 0/1, off/on, false/true
- Combinations of bits give
 - Byte = 8 bits
 - Kilobyte (KB) = $2^{10} = 1024 \approx 10^3$ bytes
 - Megabyte (MB) = 2^{10} KB = $2^{20} \approx 10^6$ bytes
 - Gigabyte (GB) = 2^{10} MB = 2^{20} KB = $2^{30} \approx 10^9$ bytes
 - Terabyte (TB) = 2^{10} GB = 2^{20} MB = 2^{30} KB = $2^{40} \approx 10^{12}$ bytes
 - Petabyte (PB) = 2^{10} TB = $\dots = 2^{40}$ KB = $2^{50} \approx 10^{15}$ bytes
- Useful: $2^{10} = 1024 \approx 10^3$

Example (Big data)

- **SKA**: Square Kilometre Array <http://www.ska.gov.au> will generate enough raw data to fill 15 million 64 GB iPads every day!
- **Sirca**: <http://www.sirca.org.au/> Global financial data for members

Arithmetic operations

- **Unary operators:** take a single argument.
 - Examples: $+3$, -3
 - Mathematics: $4!$ (factorial), $\sqrt{3}$ (square root), 3^2 (square), A^T (transpose)
- **Binary operators:** take two arguments.
 - Examples: $3 + 5$, $3 - 5$, $3 * 5$, $3 / 5$, $3 \wedge 5$
- **Precedence of arithmetic operators**
 - 1 Parentheses ()
 - 2 Powers $\wedge$
 - 3 Multiplication $*$ and division $/$
 - 4 Addition $+$ and subtraction $-$
 - 5 Operators of the same precedence are evaluated **left to right**
- Try the following operations in MATLAB, Python, Excel, or your calculator:
 - -3^2 , $0-3^2$
 - $2^{-3/2}$, $2^{(-3/2)}$, $(2^{-3})/2$
 - 3^2*2 , $3^2/2$, $12/2*3$, $1/60*60$, $1/60/60$

Numbers on a computer

- Computers store numbers in binary (base 2) and perform arithmetic on those binary representations.
- **Integers**: sign and binary digits.
 - Integers are represented exactly if they lie within the representable range.
 - Largest **signed** 32-bit integer (on modern computers): with 32 bits, 2^{32} different values can be represented, ranging from -2^{31} to $2^{31} - 1$. Hence, the largest value is $2^{31} - 1 = 2147483647 \approx 2.1 \times 10^9$.
Note: In two's complement, which is standard on modern computers, **negative** numbers are obtained by inverting all bits (of the **positive** number written in the same fixed width) and then adding 1.
 - Largest **signed** 64-bit integer is $2^{63} - 1 \approx 9.2 \times 10^{18}$.
 - MATLAB has fixed-width integer types such as **int32** and **int64**; Python's built-in **int** is arbitrary precision.
- **Floating-point numbers (FPNs)**: approximate real numbers and are not evenly spaced.
 - FPNs consist of a sign, a fraction (mantissa), and an exponent.
 - Standard types: single precision (**32 bits**) & double precision (**64 bits**).

Storage

Example (Storing a matrix)

What is the largest n -by- n matrix that can be stored in 512 KB cache, assuming each element requires 8 bytes?

Solution

Storage required for n -by- n matrix is equal to $8n^2$ bytes. Thus,

$$8n^2 = 512 \times 2^{10} = 2^{19} \implies n^2 = 2^{16} \implies n = 2^8 = 256.$$

Memory

Example (Memory)

What is the largest amount of memory (RAM) that can be addressed on a

- 1 32-bit pointer (32-bit operating system)
- 2 64-bit pointer (64-bit operating system)

Solution

- 1 A 32-bit pointer can represent an **unsigned** integer from 0 to $2^{32} - 1$. Each byte of memory has a unique address, represented by such a pointer. Therefore, it can address 2^{32} bytes of memory:

$$2^{32}\text{bytes} = 2^{22}\text{KB} = 2^{12}\text{MB} = 4\text{GB}$$

- 2 Similarly, a 64-bit pointer can address 2^{64} bytes of memory:

$$2^{64}\text{bytes} = 2^{34}\text{GB} = 16\text{Exabytes}$$

Representing real numbers

- Decimal representation

$$452.905 = (452.905)_{10}$$

$$= 4 \times 10^2 + 5 \times 10^1 + 2 \times 10^0 + 9 \times 10^{-1} + 0 \times 10^{-2} + 5 \times 10^{-3}$$

- Non-normalized forms

$$0.452905 \times 10^3 = 0.0452905 \times 10^4 = 45.2905 \times 10^1$$

- Normalized form (one digit before decimal point, and it's nonzero)

$$4.52905 \times 10^2$$

- Binary representation

$$\begin{aligned}(1001.11101)_2 &= (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) \\ &\quad + (1 \times 2^{-2}) + (1 \times 2^{-3}) + (0 \times 2^{-4}) + (1 \times 2^{-5}) \\ &= 8 + 1 + 0.5 + 0.25 + 0.125 + 0.0625 = (9.90625)_{10}\end{aligned}$$

$$(9.90625)_{10} = (1001.11101)_2 = (1.00111101)_2 \times 2^{(11)_2} \quad (\text{normalized})$$

Representing real numbers (continued)

Example

Convert 6.25 to binary.

$$(6.25)_{10} = 6 + 0.25 = 2^2 + 2^1 + 2^{-2} = (110)_2 + (0.01)_2 = (110.01)_2.$$

So 6.25 can be represented exactly in binary.

Remark

Now consider $1/10$. To convert 0.1 to binary, repeatedly multiply the fractional part by 2:

$$0.1 \times 2 = 0.2 \quad \Rightarrow \quad 0$$

$$0.2 \times 2 = 0.4 \quad \Rightarrow \quad 0$$

$$0.4 \times 2 = 0.8 \quad \Rightarrow \quad 0$$

$$0.8 \times 2 = 1.6 \quad \Rightarrow \quad 1, \text{ remainder } 0.6$$

$$0.6 \times 2 = 1.2 \quad \Rightarrow \quad 1, \text{ remainder } 0.2$$

The remainders now repeat, so $1/10 \approx (0.0001100110011\dots)_2$. Hence it has an infinite binary expansion and **cannot** be represented exactly on a binary computer.

Floating-point numbers (FPNs)

Definition

FPNs are **computer approximations** to real numbers. A normalized binary FPN has the form: with $d_j \in \{0, 1\}$ for $j = 1, \dots, p$ for some $p > 0$,

$$\pm(1.d_1d_2\dots d_p)_2 \times 2^e, \quad e_{\min} \leq e \leq e_{\max}.$$

In double precision (64 bits altogether):

- 1 bit for the sign ($\pm$)
- 52 bits for the fraction (mantissa), so $p = 52$
- 11 bits for the exponent field, which stores values from $0 = (00000000000)_2$ to $2047 = (11111111111)_2 = \sum_{i=0}^{10} 2^i$.

Note: The exponent field is stored with bias **1023**, so the exponent ranges from -1023 to **1024**. This allows both negative and positive exponents, which is needed to represent very small and very large positive numbers.

FPNs, IEEE: Institute of Electrical and Electronics Engineers, IEEE 754 standard

- However, in practice, for double precision,
 - The largest exponent is 1023 and the smallest exponent is -1022
 - This leaves room to represent special values such as 0, subnormal numbers (allowing gradual underflow), $\pm\infty$, and NaN.
- Characteristics of FPNs

- **Relative machine precision** ϵ is the gap between 1 and the next normalized FPN (> 1). For IEEE double precision,

$$\epsilon = (1.000\dots001)_2 \times 2^0 - 1 = (1 + 2^{-52}) - 1 = 2^{-52} \approx 2.22 \times 10^{-16}.$$

Note: If ϵ_i denotes the gap between the positive integer i and the next normalized FPN ($> i$), then

$$\epsilon_i = 2^m \epsilon, \quad \text{for } 2^m \leq i < 2^{m+1}, \text{ with } m \geq 0.$$

ϵ : MATLAB `eps`, Python `sys.float_info.epsilon` (`import sys`)

- **Largest finite FPN** $= (2 - \epsilon) \times 2^{1023}$ (MATLAB `realmax`, Python `sys.float_info.max`)
- **Smallest positive normalized FPN** $= 2^{-1022}$ (MATLAB `realmin`, Python `sys.float_info.min`)

FPNs, IEEE extensions: inf , NaN

Definition (Infinity and Not a Number)

- **Overflow**: any number much larger than the largest finite FPN is represented by inf ; any number much smaller than the most negative finite FPN is represented by $-\text{inf}$.
 - **Not a Number**: any result that is mathematically undefined is represented by NaN .
-
- inf behaves like ∞ and propagates through computations
 - Evaluate 1.8×10^{308} , -1.8×10^{308}
 - Evaluate $2.4/0$, $-3.6/0$
 - Evaluate $\pi + \text{inf}$, $3.6 * \text{inf}$
 - NaN propagates
 - Evaluate $0/0$, $\text{inf} - \text{inf}$, inf/inf
 - Evaluate $9.3 * \text{NaN}$, $\sin(\text{NaN})$

Try Python: `from numpy import inf; d = inf; 3/d`

FPNs, Underflow

Underflow: A number smaller than the smallest positive normalized FPN may be represented as a subnormal number; if it is even smaller, it **underflows** to 0.

Example (Gradual underflow)

- Find x such that $e^{-x} = \text{realmin}$ (MATLAB) or `sys.float_info.min` (Python)
- Calculate $f(x) = e^{-x}$ for $x = 700, 701, \dots, 750$

Solution. $x \approx -\log(2.22 * 10^{-308}) \approx 708.4$

MATLAB

```
x=700:750;
exp(-x);
```

Python

```
from numpy import exp, arange
x=arange(700,751);
exp(-x)
```

x	e^{-x} , MATLAB	e^{-x} , Python
707	9.0e-308	9.0e-308
708	3.3e-308	3.3e-308
709	1.2e-308	1.2e-308
710	4.5e-309	4.5e-309
742	5.4e-323	5.4e-323
743	2.0e-323	2.0e-323
744	9.9e-324	9.9e-324
745	4.9e-324	0
746	0	0
747	0	0

FPNs, a simple example

Example

Describe the FPN system with $p = 2$, $e_{\max} = 2$ and $e_{\min} = -1$.

$$\pm(1.d_1d_2)_2 \times 2^e, \quad -1 \leq e \leq 2, \quad d_1, d_2 \in \{0, 1\}.$$

We can represent the following binary numbers: $\pm$

$$\begin{aligned} &(1.00)_2 \times 2^{-1}, \quad (1.01)_2 \times 2^{-1}, \quad (1.10)_2 \times 2^{-1}, \quad (1.11)_2 \times 2^{-1}, \\ &(1.00)_2 \times 2^0, \quad (1.01)_2 \times 2^0, \quad (1.10)_2 \times 2^0, \quad (1.11)_2 \times 2^0, \\ &(1.00)_2 \times 2^1, \quad (1.01)_2 \times 2^1, \quad (1.10)_2 \times 2^1, \quad (1.11)_2 \times 2^1, \\ &(1.00)_2 \times 2^2, \quad (1.01)_2 \times 2^2, \quad (1.10)_2 \times 2^2, \quad (1.11)_2 \times 2^2. \end{aligned}$$

In decimals: $\pm$

$$0.5, 0.625, 0.75, 0.875, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 5, 6, 7.$$

Is the order of addition and subtraction important?

Balance sheet example

- An accountant prepares a balance sheet of cash flows:
 - Initial balance B_0
 - Receipts R_k and expenses/costs C_k , for each time period $k = 1, \dots, T$
 - Final balance B_T at time period T
- Calculate the balance B_T
 - ① $B_k = B_{k-1} + R_k - C_k$ for $k = 1, \dots, T$
 - ② $\bar{B}_T = B_0 + \sum_{k=1}^T R_k - \sum_{k=1}^T C_k$
 - ③ $\hat{B}_T = B_0 + \sum_{k=1}^T (R_k - C_k)$
- Is
 - $\bar{B}_T = \hat{B}_T$?
 - $\bar{B}_T - \hat{B}_T = 0$?
- Computer arithmetic does **not** obey the same rules (**associativity**) as mathematics
- Testing for exact equality when dealing with FPNs is a **bad idea**
- MATLAB M-file **balances.m**; Python file **balances.py**

Balance sheet example, MATLAB script

```
% MATH3311/MATH5335: File = balances.m
%
% Chk1: Is final balance B(end) equal to
%       B0 + sum of receipts - sum of costs ?
% Chk2: Is 1361.30 - 961.20 equal to 400.10 ?
% Why are Chk1 and Chk2 false (0) not true (1)
```

```
format short
```

```
B0 = 0;
R = [1234.56; 0; 1361.30];
C = [0; 1234.56; 961.20];
B = B0 + cumsum(R - C);
```

```
Table = [R C B]
Bend = B(end), Bfin = B0 + sum(R) - sum(C)
Chk1 = B(end) == Bfin
Chk2 = 1361.30 - 961.20 == 400.10
```

```
% Numerical values of differences
%format short e
Chk1n = Bend - Bfin
Chk2n = (1361.30 - 961.20) - 400.10
```

Balance sheet example, MATLAB results

balances

Table =

1234.56	0	1234.56
0	1234.56	0
1361.30	961.20	400.10

Bend =

400.10

Bfin =

400.10

Chk1 =

0

Chk2 =

0

Chk1n =

0.00

Chk2n =

-0.00

diary off

Balance sheet example, Python script

```
1 # MATH3311/MATH5335: File = balances.py
2 from numpy import array, empty, cumsum, sum
3 B0 = 0
4 R = array([1234.56, 0, 1361.30])
5 C = array([0, 1234.56, 961.20])
6 B = B0 + cumsum(R - C)
7 Table = empty((3,3))
8 Table[:,0] = R
9 Table[:,1] = C
10 Table[:,2] = B
11 print(Table)
12 Bend = B[-1]
13 Bfin = B0 + sum(R) - sum(C)
14 print(f'Bend={Bend:0.2f}')
15 print(f'Bfin={Bfin:0.2f}')
16 Chk1 = (B[-1] == Bfin)
17 Chk2 = (1361.30 - 961.20 == 400.10 )
18 print('Chk1=', Chk1, 'Chk2=', Chk2)
19 Chk1n = round(Bend - Bfin,6)
20 Chk2n = round((1361.30 - 961.20) - 400.10,6)
```

Errors and number of significant figures

Let $\bar{x}$ be a computed approximation to x .

- The **absolute error** is $e_a = |\bar{x} - x|$
- For $x \neq 0$, the **relative error** is $e_r = |\bar{x} - x|/|x|$
- The **number of significant decimal figures** (nsf) is the largest positive integer p such that $e_r < (1/2) 10^{-p}$

Example

- 1 Calculate the e_r and the nsf in using $22/7$ as an approximation to π .

$$e_r = |\pi - 22/7|/\pi \approx 4.0250 \times 10^{-4} < 5 \times 10^{-4} = (1/2) \times 10^{-3}.$$

Thus, the number of significant figures $p = 3$.

Note: $22/7 = 3.1428\dots$ and $\pi = 3.1415\dots$ (3 correct significant figures)

- What error is acceptable in your computation?
 - 10% or relative error of 0.1?
 - 6 significant figures, that is, $e_r \leq 0.5 \times 10^{-6}$?
 - Full machine precision?

Rounding error

Definition (Rounding error)

The **rounding error** in storing x on a computer is the absolute error between the exact value of x and the stored value.

Estimating rounding error

- ϵ is the relative machine precision on a computer
- $\epsilon|x|$ gives an estimate (or a bound) of the rounding error in storing x on a computer.

Example (Rounding error in double precision)

Estimate the rounding error in storing the value $x = 0.12$.

The rounding error is $\epsilon|x| \approx 2.2 \times 10^{-16} \times 0.12$.

Catastrophic cancellation

Definition (Catastrophic cancellation)

If $\bar{x}$ and $\bar{y}$ are computed approximations to nonzero quantities x and y , with $x \approx y$, then the subtraction $\bar{x} - \bar{y}$ may lose many significant digits. In particular, the relative error in $\bar{x} - \bar{y}$ can be much larger than the relative errors in $\bar{x}$ and $\bar{y}$.

- Subtracting nearly equal quantities is dangerous because the leading digits cancel.
- This often produces a result with much lower accuracy than the original data.
- See the next two examples for a typical numerical illustration.

Catastrophic cancellation (continued)

Example (Catastrophic cancellation)

Suppose $b = 1.12345$ and $c = 1.12351$ are measured to 6 significant figures. Estimate the relative error in $b - c$.

Solution

Relative error in b : $\left| \frac{b - \bar{b}}{b} \right| < \frac{10^{-6}}{2}$. Absolute error in b : $|b - \bar{b}| < \frac{10^{-6}}{2} b$.

Relative error in c : $\left| \frac{c - \bar{c}}{c} \right| < \frac{10^{-6}}{2}$. Absolute error in c : $|c - \bar{c}| < \frac{10^{-6}}{2} c$.

Relative error in $b - c$:

$$\begin{aligned} \left| \frac{(b - c) - (\bar{b} - \bar{c})}{b - c} \right| &\leq \left| \frac{b - \bar{b}}{b - c} \right| + \left| \frac{c - \bar{c}}{b - c} \right| < \frac{10^{-6}}{2} \frac{b + c}{c - b} \\ &= \frac{10^{-6}}{2} \frac{2.24696}{6 \times 10^{-5}} \approx 0.1872 \times 10^{-1}. \end{aligned}$$

One significant figure can be guaranteed only.

Catastrophic cancellation (continued)

Example (Roots of a quadratic)

The roots of the quadratic $ax^2 + bx + c$ are $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

- ① When is this formula susceptible to catastrophic cancellation?
- ② Find a mathematically equivalent formula that avoids this difficulty.

Solution

- ① When $4ac$ is much smaller than b^2 in magnitude, we have $\sqrt{b^2 - 4ac} \approx |b|$. Say $b > 0$, then $-b + \sqrt{b^2 - 4ac} \approx 0$. The quadratic formula is susceptible to catastrophic cancellation.
- ② When $b > 0$, a stable equivalent formula is obtained by multiplying by the conjugate:

$$\frac{-b + \sqrt{b^2 - 4ac}}{2a} = \frac{-b^2 + (b^2 - 4ac)}{2a(b + \sqrt{b^2 - 4ac})} = -\frac{2c}{(b + \sqrt{b^2 - 4ac})}$$

Floating-point operations – flops

Definition (Floating-point operation)

A **flop** is one of the basic operations $+$, $-$, $*$, $/$ applied to **FPNs**.

- **flops** and **flops/sec** (floating-point operations per second) are used to **estimate** the time a task will take.
- **Assume** $+$, $-$, $*$, $/$ take the same time. In reality $*$, $/$ usually take longer than $+$, $-$
- **Performance units**
 - **Mflops**, Megaflops = 10^6 flops
 - **Gflops**, Gigafllops = 10^9 flops
 - **Tflops**, Teraflops = 10^{12} flops
- **Speed (flops/sec) = Clock (cycles/sec) $\times$ Cores $\times$ flops/cycle**
 - Clock rate: in GHz, that is, 10^9 cycles per second
 - Cores: CPUs with 4–16 cores, GPUs with thousands of cores, or **Many Integrated Core** (MIC) coprocessor with 50–80 cores
 - flops/cycle: modern CPUs can do 4 or more

Estimating computation time

Example (Matrix multiplication)

Suppose a 3 GHz quad core PC can do 2 flops per clock cycle. Matrix multiplication $C = AB$ where $A, B, C \in \mathbb{R}^{n \times n}$ is defined by

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj} \quad i = 1, \dots, n, \quad j = 1, \dots, n$$

- 1 How many flops are required to calculate C
- 2 Estimate how long it will take to calculate C for $n = 5000$.
- 3 Estimate the largest matrices that can be multiplied in 1 hour.

Estimating computation time (continued)

Solution

① Flops for matrix multiplication

- Matrix multiplication requires n multiplications and $n - 1$ additions for each element C_{ij} .
- Since there are n^2 elements C_{ij} , this gives n^3 multiplications and $n^2(n - 1)$ additions.
- Therefore, the total number of flops is $n^2(2n - 1) \approx 2n^3$.

② Time for $n = 5000$

- Speed = clock rate $\times$ cores $\times$ flops/clock cycle = flops/sec.
- Speed = $3 \times 10^9 \times 4 \times 2 = 24 \times 10^9$ flops/sec.
- Matrix multiplication requires approximately $2n^3 = 2.5 \times 10^{11}$ flops.
- Time = (total flops)/(flops/sec) = $2.5 \times 10^{11} / 2.4 \times 10^{10} \approx 10$ sec.

③ Largest matrix multiplication in 1 hour

- 1 hour = $60 \times 60 = 3600$ seconds.
- In 1 hour, the machine can do $3600 \times 2.4 \times 10^{10} = 8.64 \times 10^{13}$ flops.
- Solve $2n^3 = 8.64 \times 10^{13}$, so $n = (4.32 \times 10^{13})^{1/3} \approx 3.5 \times 10^4$.
- So n is approximately 35,000.



Patriot missile disaster

Gulf War: Patriot missiles were designed to shoot down incoming Scud missiles.

- Decimal vs. binary:

$$\frac{1}{10} = \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^8} + \frac{1}{2^9} + \dots = 0.0001100110011001100110011\dots$$

- A 24-bit timer stored only **0.00011001100110011001100**.
- The error was **0.00000000000000000000000110011... $\approx 9.5 \times 10^{-8}$** .
- After running for 100 hours, the accumulated error was

$$9.5 \times 10^{-8} \times 100 \times 60 \times 60 \times 10 \approx 0.34 \text{ sec.}$$

- Since the Scud moved at about 1676 metres/sec, this error became more than 0.5 km.
- The target was outside the Patriot tracking range, with tragic consequences.

References

- IEEE 754: Standard for Binary Floating Point arithmetic,
<http://grouper.ieee.org/groups/754/>
- David Goldberg, What every Computer Scientist should know about floating point arithmetic,
<http://www.validlab.com/goldberg/paper.ps>
- William Kahan, <http://www.cs.berkeley.edu/~wkahan> and
<http://www.cs.berkeley.edu/~wkahan/ieee754status/ieee754.ps>
- Thomas Huckle, Collection of Software Bugs,
<http://www5.in.tum.de/~huckle/bugse.html>
- The Risks Digest, <http://catless.ncl.ac.uk/Risks/>
- Wikipedia, Software Bug,
http://en.wikipedia.org/wiki/Computer_bug